

LangGraph State Reducers: Merge Semantics, by Hand

Why `add_messages` must be a *monoid* and a plain field must not — the algebra of how node updates fold into state, counted message by message.

What this document does. It takes the one tiny `AgentState` from Module 2.4 — the typed dict with `messages: Annotated[list, add_messages]` alongside a plain `status` field — and works out, by hand, exactly how a partial update returned by a node gets merged into the running state. We treat each field's *reducer* as an algebraic operation, fold a four-node ReAct cycle through both of them, and watch the message count grow. Every count, every merged list, and the associativity checks below are reproduced by the small Python program in Appendix A, so the arithmetic is exact and self-consistent.

Where this sits. This is **Topic 1 of Module 2.4** — the fifth of the module's five primitives (State, Nodes, Edges, Conditional Edges, *Reducers*), and the one that makes the other four safe under cycles and branches. Its companions: Module 0.0 (the ReAct loop as token accounting — the *same* growing transcript, here seen as a growing message list) and Track 4 (production agents, where a wrong reducer is a silent state-corruption bug).

Concepts covered, in order: a reducer as a binary fold · REPLACE (last-write-wins) · APPEND as list concatenation · why APPEND is a monoid (identity + associativity) · why a monoid makes parallel/branch merges well-defined · why REPLACE is order-sensitive (the bug source) · `add_messages` as upsert-by-id · message count = $\sum m_i$ over a ReAct cycle · REPLACE vs APPEND diverging on the same stream.

Internal learning material. Numbers are reproducible from Appendix A. v1.0

1 The setup: one state, two fields, two reducers

In LangGraph a node never mutates state. It *returns* a partial dict of updates, and the runtime folds that update into the current state *one field at a time*, using a per-field function called a **reducer**. The state from Module 2.4 has exactly two interesting fields:

```
1 class AgentState(TypedDict):
2     messages: Annotated[list[BaseMessage], add_messages] # reducer = APPEND
3     status: Literal["running", "awaiting_approval", "done"] # default = REPLACE
```

Write s_t for the value of a field after t node updates have been folded in, and u_t for the update the t -th node returns for that field. A reducer is the binary operation ρ that does the fold:

$$s_t = \rho(s_{t-1}, u_t), \quad s_0 = \text{initial value.} \tag{1}$$

The entire run is just this fold repeated. Two reducers appear in the state above, and they are *different algebraic objects* — that difference is the whole topic.

Quantity	Symbol	Meaning here
Field value after t updates	s_t	result of folding t node outputs
The t -th node's update	u_t	partial dict returned by a node
REPLACE reducer	$\rho_R(o, n) = n$	last write wins (plain fields)
APPEND reducer	$\rho_A(o, n) = o ++ n$	list concat (<code>add_messages</code>)
Identity of APPEND	$\varepsilon = []$	the empty list
Messages added by node i	m_i	length of that node's message update
Nodes in the cycle	N	4

Notation. `++` is list concatenation. A message is a pair $(id, text)$; `add_messages` keys on the id . We write a list as $[A, B]$ using its texts when ids and texts coincide. “Monoid” means a set with an associative binary operation and an identity element — nothing more exotic than that.

Intuition

A reducer answers one question: “a node just handed me *new*; I already hold *old*; what is the field now?” REPLACE shrugs and keeps only *new* — the past is gone. APPEND glues *new* onto the end of *old* — the past is kept, in order. Same signature $(old, new) \rightarrow value$, opposite memory. Everything below is the consequence of that one design choice, made per field.

2 Step 1 — REPLACE: last write wins, order matters

The default reducer for a plain field is REPLACE: $\rho_R(o, n) = n$. Folding it over a stream of updates simply discards everything but the final term. Unrolling Equation (1) for the `status` field over four updates:

$$s_4 = \rho_R(\rho_R(\rho_R(\rho_R(s_0, u_1), u_2), u_3), u_4) = u_4.$$

The state “tracks the latest” and nothing else. This is correct and desirable for a scalar like `status` — you want the *current* status, not a pile of old ones. But notice what it costs you: ρ_R is **not associative with history**, and more sharply, it is *order-sensitive*. If two branches finish and one wrote `from_L` while the other wrote `from_R`, the surviving value depends purely on which merge happened last:

$$\rho_R(\rho_R(\text{init}, \text{from_L}), \text{from_R}) = \text{from_R}, \quad \rho_R(\rho_R(\text{init}, \text{from_R}), \text{from_L}) = \text{from_L}.$$

Mini-example — this operation in isolation

Fold REPLACE over the status stream `running`, `running`, `awaiting_approval`, `done`: $s_1 = \text{running}$, $s_2 = \text{running}$, $s_3 = \text{awaiting_approval}$, $s_4 = \text{done}$. Only the last write, `done`, survives — the three earlier statuses left no trace. ✓ That is exactly what you want for a single-valued field, and exactly what you do *not* want for a transcript.

3 Step 2 — APPEND is a monoid, and why that matters

The `messages` field uses `add_messages`, whose core is list concatenation: $\rho_A(o, n) = o ++ n$. This operation has two algebraic properties `REPLACE` lacks.

(i) **Identity.** The empty list $\varepsilon = []$ satisfies $\varepsilon ++ L = L = L ++ \varepsilon$ for every list L . A node that returns no messages changes nothing — the natural “no-op”.

(ii) **Associativity.** For any lists,

$$(L_1 ++ L_2) ++ L_3 = L_1 ++ (L_2 ++ L_3).$$

A set with an associative operation and an identity *is* a monoid. So `add_messages` is the free monoid on messages; `REPLACE` is not a monoid (it has no identity and forgets its left argument).

Why care? Because `LangGraph` may run branches in parallel and merge their updates, and the runtime does not promise an order. If the merge operation is associative, then $(\text{base} ++ L) ++ R = \text{base} ++ (L ++ R)$ — regrouping the branches *cannot change the result*. The merged transcript is well-defined no matter how the engine schedules the joins.

APPEND associative $\implies$ branch-merge order is irrelevant $\implies$ merge is well-defined.

 (2)

`REPLACE` has the opposite property, shown in Step 1: regrouping flips the answer. That is the bug. Putting a scalar reducer on a field two branches both write is how agents silently lose one branch’s update — and which one they lose depends on a race.

Mini-example — this operation in isolation

Take `base` = `[B1]`, branch `L` = `[L1, L2]`, branch `R` = `[R1]`. Merge left-first: $([B1] ++ [L1, L2]) ++ [R1] = [B1, L1, L2, R1]$. Merge with `L, R` grouped first: $[B1] ++ ([L1, L2] ++ [R1]) = [B1, L1, L2, R1]$. **Identical.** ✓ The same two branches under a `REPLACE` scalar give `from_R` one way and `from_L` the other — two different answers from the same inputs.

Watch out

“Append” here means *concatenate*, not “commute”. `APPEND` is associative but **not** commutative: $L ++ R \neq R ++ L$ in general. Associativity is what makes the merge well-defined; it does *not* promise that two parallel branches land in a canonical relative order. `add_messages` resolves that separately by keying on message *id* (next step). Do not read “monoid” as “order-free.”

4 Step 3 — add_messages is upsert-by-id, not blind append

`add_messages` is concatenation with one refinement: it keys on each message’s *id*. If an incoming message carries an *id* already present, it *updates that entry in place* rather than appending a duplicate. This is an **upsert** (update-or-insert) keyed by *id*:

$$\rho_A(o, n)_{\text{upsert}} : \quad \text{for each } (k, v) \in n : \begin{cases} \text{replace the entry with id } k & \text{if } k \text{ is already in } o, \\ \text{append } (k, v) \text{ at the end} & \text{otherwise.} \end{cases}$$

It stays a monoid over *distinct* ids (new ids append, identity is still `[]`); the upsert only changes what happens on a key collision. The module’s own state comment names exactly this: “`add_messages` (appends & de-dupes).”

Mini-example — this operation in isolation

Start $[A, B]$. A node adds $[C]$ (new id) $\Rightarrow [A, B, C]$. A node then emits B' with the *same* id as $B \Rightarrow$ the entry for B is overwritten in place, giving $[A, B', C]$ — length stays 3, no duplicate B . ✓ A blind append would have produced the wrong $[A, B, C, B']$ with a stale B still lurking.

Counting growth across a ReAct cycle. On the *happy path* (all ids distinct — think, act, observe, finish), no upsert collisions fire, so `add_messages` is pure append and the message count is just the running sum of what each node added. If node i contributes m_i messages, then after N nodes

$$|\text{messages}| = \sum_{i=1}^N m_i. \quad (3)$$

For a 4-node ReAct cycle — **agent** (a thought + an action = 2), **tools** (one observation = 1), **agent** again (2), **final** (one answer = 1) — we get $m = (2, 1, 2, 1)$:

Node	What it does	m_i	count after	running $\sum m_i$
1	agent (thought + action)	2	2	2
2	tools (observation)	1	3	3
3	agent (thought + action)	2	5	5
4	final (answer)	1	6	6
Total		6	6	6

The “count after” column equals the running-sum column at every row — the fold of APPEND literally *is* the partial-sum sequence of m_i . ✓ Total = $2 + 1 + 2 + 1 = 6 = \sum m_i$, matching Equation (3).

5 Step 4 — REPLACE vs APPEND on the same stream, side by side

Now run *one* stream of node updates through *both* reducers at once — the exact situation inside `AgentState`, where `status` folds with REPLACE while `messages` folds with APPEND. Each node i returns a `status` value and one message:

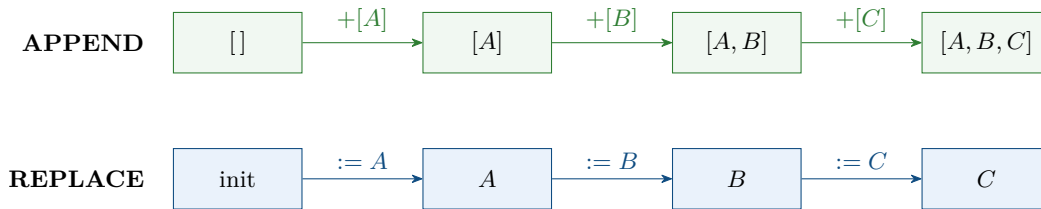
Node	update returned	REPLACE status	APPEND messages
1	status="running", +[u1]	running	1
2	status="running", +[u2]	running	2
3	status="awaiting_approval", +[u3]	awaiting_approval	3
4	status="done", +[u4]	done	4

The two columns **diverge by construction**. REPLACE collapses four writes into one surviving scalar, **done** — it holds $O(1)$ information regardless of how many nodes ran. APPEND retains all four, $|\text{messages}| = 4 = \sum 1$ — it holds $O(N)$ information. ✓ Same inputs, same stream order, completely different memory: that is the reducer choice deciding the shape of your state.

Intuition

Pick the reducer by asking “do I want the *latest* or the *whole history*?” A status flag, a current plan, a retry counter — latest, so REPLACE. A conversation, a list of tool observations, an audit trail — history, so an append-style monoid. Getting this backwards is the classic LangGraph bug: put REPLACE on `messages` and every node *overwrites* the transcript, so the agent “forgets” everything before the last node and the LLM gets a one-message context. The fix is one annotation, `Annotated[list, add_messages]`.

6 The fold, drawn



Same three updates A, B, C , two reducers. The green APPEND lane *grows* — each arrow keeps the past; the final state is the whole stream. The blue REPLACE lane stays length one — each arrow overwrites; the final state is only the last update. The picture *is* Equation (1) read with ρ_A on top and ρ_R below.

7 Reducer cheat-sheet

The fold	$s_t = \rho(s_{t-1}, u_t), \quad s_0 = \text{initial}$
REPLACE (default)	$\rho_R(o, n) = n \quad (\text{last write wins})$
APPEND (<code>add_messages</code>)	$\rho_A(o, n) = o ++ n \quad (\text{list concat})$
Identity of APPEND	$\varepsilon = [], \quad \varepsilon ++ L = L = L ++ \varepsilon$
Associativity	$(L_1 ++ L_2) ++ L_3 = L_1 ++ (L_2 ++ L_3)$
$\Rightarrow$ monoid	associative + identity = merge well-defined
Upsert-by-id	collide on <i>id</i> $\rightarrow$ update in place; else append
Count after N nodes	$ \text{messages} = \sum_{i=1}^N m_i \quad (\text{distinct ids})$
REPLACE info content	$O(1)$ — only the latest write
APPEND info content	$O(N)$ — the full ordered history

8 An honest word about these numbers

The specific counts here ($m = (2, 1, 2, 1)$, total 6; the four-write status stream) are illustrative round choices, picked so the fold is followable on paper — a real ReAct turn might emit a different number of messages, and a long run produces hundreds. What is *not* illustrative is the algebra: REPLACE is $\rho(o, n) = n$, forgetful and order-sensitive; APPEND is a monoid, so its branch merges are well-defined regardless of scheduling; and `add_messages` is that monoid plus an upsert-by-id, so the happy-path count is always exactly $\sum m_i$. Change the message counts and the table moves; the monoid laws, the upsert rule, and the REPLACE-vs-APPEND divergence do not. That structure is the thing to carry into every graph you build — it is why `messages` gets an annotation and `status` does not.

Appendix A — Reference implementation

Every count and every merged list in this document is produced by the program below. It implements both reducers as plain folds, then reproduces the upsert example, the 4-node count growth, the REPLACE-vs-APPEND divergence, and the monoid checks.

```

1  # reducers.py -- reproduces every number in "LangGraph State Reducers".
2
3  def replace(old, new): # default reducer: last write wins
4      return new
5
6  def add_messages(old, new): # APPEND with upsert-by-id
7      merged = list(old)
8      index = {mid: i for i, (mid, _) in enumerate(merged)}
9      for mid, text in new:
10         if mid in index:
11             merged[index[mid]] = (mid, text) # update in place (upsert)
12         else:
13             index[mid] = len(merged)
14             merged.append((mid, text)) # append new id
15     return merged
16
17 # Part 1 -- upsert: start [A,B]; add [C]; update B' (same id) -> [A,B',C]
18 s = [("A", "A"), ("B", "B")]
19 s = add_messages(s, [("C", "C")])
20 s = add_messages(s, [("B", "B'")])
21 print("upsert result:", [t for _, t in s], " count:", len(s))
22
23 # Part 2 -- message-count growth over a 4-node ReAct cycle
24 m = [2, 1, 2, 1] # agent, tools, agent, final
25 msgs, running = [], 0
26 for i, mi in enumerate(m, 1):
27     msgs = add_messages(msgs, [(f"n{i}_{k}", f"m{i}_{k}") for k in range(mi)])
28     running += mi
29     print(f"node {i}: m={mi} count={len(msgs)} running={running}")
30 assert len(msgs) == sum(m)
31 print("total =", len(msgs), " sum(m_i) =", sum(m))
32
33 # Part 3 -- REPLACE vs APPEND on the same stream
34 updates = [("running", [("u1", "x")]), ("running", [("u2", "y")]),
35            ("awaiting_approval", [("u3", "z")]), ("done", [("u4", "w")])]
36 status, acc = "init", []
37 for i, (st, add) in enumerate(updates, 1):
38     status = replace(status, st)
39     acc = add_messages(acc, add)
40     print(f"node {i}: REPLACE={status:<18} APPEND_len={len(acc)}")
41 print("final REPLACE:", status, " final APPEND_len:", len(acc))

```

```

42
43 # Part 4 -- monoid checks: associativity, identity; REPLACE order-sensitivity
44 base, L, R = [("b1", "B1")], [("l1", "L1"), ("l2", "L2")], [("r1", "R1")]
45 lhs = add_messages(add_messages(base, L), R)
46 rhs = add_messages(base, add_messages(L, R))
47 print("APPEND associative:", [t for _, t in lhs] == [t for _, t in rhs])
48 print("APPEND identity:", add_messages([], L) == L and add_messages(L, []) == L)
49 print("REPLACE order-sensitive:",
50       replace(replace("init", "from_L"), "from_R")
51       != replace(replace("init", "from_R"), "from_L"))
52
53 # Expected output:
54 # upsert result: ['A', 'B', 'C'] count: 3
55 # node 1: m=2 count=2 running=2
56 # node 2: m=1 count=3 running=3
57 # node 3: m=2 count=5 running=5
58 # node 4: m=1 count=6 running=6
59 # total = 6 sum(m_i) = 6
60 # node 1: REPLACE=running APPEND_len=1
61 # node 2: REPLACE=running APPEND_len=2
62 # node 3: REPLACE=awaiting_approval APPEND_len=3
63 # node 4: REPLACE=done APPEND_len=4
64 # final REPLACE: done final APPEND_len: 4
65 # APPEND associative: True
66 # APPEND identity: True
67 # REPLACE order-sensitive: True

```

— end of document —